Smith–Waterman algorithm

From Wikipedia, the free encyclopedia

The **Smith–Waterman algorithm** performs local sequence alignment; that is, for determining similar regions between two strings or nucleotide or protein sequences. Instead of looking at the total sequence, the Smith–Waterman algorithm compares segments of all possible lengths and optimizes the similarity measure.

The algorithm was first proposed by Temple F. Smith and Michael S. Waterman in 1981.[1] Like the Needleman–Wunsch algorithm, of which it is a variation, Smith–Waterman is adynamic programming algorithm. As such, it has the desirable property that it is guaranteed to find the optimal local alignment with respect to the scoring system being used (which includes the substitution matrix and the gap-scoring scheme). The main difference to the Needleman–Wunsch algorithm is that negative scoring matrix cells are set to zero, which renders the (thus positively scoring) local alignments visible. Backtracking starts at the highest scoring matrix cell and proceeds until a cell with score zero is encountered, yielding the highest scoring local alignment. One does not actually implement the algorithm as described because improved alternatives are now available that have better scaling(Gotoh, 1982) [2] and are more accurate (Altschul and Erickson, 1986).[3]

Contents

[hide]

- 1 Explanation
- 2 Example
- 3 Motivation
- 4 Accelerated versions
  - 4.1 FPGA
  - 4.2 GPU
  - 4.3 SIMD
  - 4.4 Cell Broadband Engine
- 5 See also
- 6 References

- 7 External links

Explanation[edit]

A matrix H is built as follows:

$$H(i,0) = 0,\ 0 \le i \le m$$

$$H(0,j) = 0,\ 0 \le j \le n$$

if $a_i = b_j$ then $w(a_i, b_j) = w(\text{match})$ or if $a_i \neq b_j$ then $w(a_i, b_j) = w(\text{mismatch})$

$$H(i,j) = \max \left\{ \begin{array}{cc} 0 & \\ H(i-1,j-1) + \ w(a_i,b_j) & \text{Match/Mismatch} \\ H(i-1,j) + \ w(a_i,-) & \text{Deletion} \\ H(i,j-1) + \ w(-,b_j) & \text{Insertion} \end{array} \right\},\ 1 \le i \le m, 1 \le j \le n$$

Where:

- a, b = Strings over the Alphabet Σ
- $m = \text{length}(a)$
- $n = \text{length}(b)$
- $H(i,j)$ – is the maximum Similarity-Score between a suffix of a[1...i] and a suffix of b[1...j]
- $w(c,d),\ c,d \in \Sigma \cup \{'-'\}$, '–' is the gap-scoring scheme

Example[edit]

- Sequence 1 = `ACACACTA`
- Sequence 2 = `AGCACACA`

- *w*(match) = +2
- $w(a,-) = w(-,b) = w(\text{mismatch}) = -1$

$$H = \begin{pmatrix} & - & A & C & A & C & A & C & T & A \\ - & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ A & 0 & 2 & 1 & 2 & 1 & 2 & 1 & 0 & 2 \\ G & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 \\ C & 0 & 0 & 3 & 2 & 3 & 2 & 3 & 2 & 1 \\ A & 0 & 2 & 2 & 5 & 4 & 5 & 4 & 3 & 4 \\ C & 0 & 1 & 4 & 4 & 7 & 6 & 7 & 6 & 5 \\ A & 0 & 2 & 3 & 6 & 6 & 9 & 8 & 7 & 8 \\ C & 0 & 1 & 4 & 5 & 8 & 8 & 11 & 10 & 9 \\ A & 0 & 2 & 3 & 6 & 7 & 10 & 10 & 10 & 12 \end{pmatrix} \qquad T = \begin{pmatrix} & - & A & C & A & C & A & C & T & A \\ - & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ A & 0 & \nwarrow & \leftarrow & \nwarrow & \leftarrow & \nwarrow & \leftarrow & \leftarrow & \nwarrow \\ G & 0 & \uparrow & \nwarrow & \uparrow & \nwarrow & \uparrow & \nwarrow & \nwarrow & \uparrow \\ C & 0 & \uparrow & \nwarrow & \nwarrow & \nwarrow & \leftarrow & \nwarrow & \leftarrow & \leftarrow \\ A & 0 & \nwarrow & \uparrow & \nwarrow & \leftarrow & \nwarrow & \leftarrow & \leftarrow & \nwarrow \\ C & 0 & \uparrow & \nwarrow & \uparrow & \nwarrow & \leftarrow & \nwarrow & \leftarrow & \leftarrow \\ A & 0 & \nwarrow & \uparrow & \nwarrow & \uparrow & \nwarrow & \leftarrow & \leftarrow & \nwarrow \\ C & 0 & \uparrow & \nwarrow & \uparrow & \nwarrow & \uparrow & \nwarrow & \leftarrow & \leftarrow \\ A & 0 & \nwarrow & \uparrow & \nwarrow & \uparrow & \nwarrow & \uparrow & \nwarrow & \nwarrow \end{pmatrix}$$

To obtain the optimum local alignment, we start with the highest value in the matrix (i,j). Then, we go backwards to one of positions ($i - 1$,j), (i, $j - 1$), and ($i - 1$, $j - 1$) depending on the direction of movement used to construct the matrix. We keep the process until we reach a matrix cell with zero value.

In the example, the highest value corresponds to the cell in position (8,8). The walk back corresponds to (8,8), (7,7), (7,6), (6,5), (5,4), (4,3), (3,2), (2,1), (1,1), and (0,0),

Once we've finished, we reconstruct the alignment as follows: Starting with the last value, we reach (i,j) using the previously calculated path. A diagonal jump implies there is an alignment (either a match or a mismatch). A top-down jump implies there is a deletion. A left-right jump implies there is an insertion.

For the example, we get:

```
Sequence 1 = A-CACACTA
Sequence 2 = AGCACAC-A
```

Motivation